THANK YOU to Our Sponsors

PLATINUM

Progress®

BizStream

meijer®

GOLD

ATOMIC OBJECT

SILVER

THE C2 GROUP

COLLEGE OF COMPUTING
GRAND VALLEY STATE UNIVERSITY®

Kentico

uno PLATFORM

LeadingEDJE

AuthorizationHub.

LAFLEUR®

Spec-Driven Development

with GitHub Spec Kit

From blank page to finished draft — building a book-writing app, one spec at a time.

Barret Blake
Architect, NimblePros
https://barretblake.dev
BlueSky: @barretblake.dev
https://linkedin/in/barretblake

WHO AM I

Barret Blake

Barret Blake

Architect · NimblePros

Microsoft® MVP Most Valuable Professional

A few things about me

- Microsoft MVP — .NET
- Husband & Father
- Coder, Blogger & Speaker
- Mentor
- Gamer
- Model Railroader & LEGO Enthusiast
- Buckeyes & Browns Fan

NimblePros
Better software development. Delivered.

ROADMAP

What We'll Cover Today

The Problem

Why “vibe coding” breaks down at scale

Spec-Driven Development

Specs as the source of truth for AI agents

Spec Kit in 2026

What the toolkit is — and what's new this year

The Workflow

Constitution → Specify → Plan → Implement

Meet InkWell

Our demo: a novelist's writing studio

Make It Yours

Extensions, presets & best practices

THE PROBLEM

Vibe Coding: Staring at a Blank Page

What is vibe coding?

Tossing vague prompts at an AI coding assistant and accepting whatever it hands back — no plan, no spec, just vibes.

Like starting a novel with no outline, no characters, no ending in mind.

The result:

- Unpredictable, inconsistent code quality
- Features that miss the actual requirement
- Hours lost debugging the AI's misunderstandings
- Over-engineered solutions nobody asked for

WHY IT HAPPENS

We Treat AI Like a Search Engine

“The issue isn't the coding agent's ability, but our approach. We treat coding agents like search engines when we should treat them more like literal-minded pair programmers.”

— Den Delimarsky, former GitHub Principal Product Manager

The key insight

AI assistants need clear, structured context and specifications — not vague wishes. They excel at execution when you give them real direction.

THE SHIFT

What Is Spec-Driven Development?

An approach where specifications become the source of truth — guiding AI agents to build exactly what's needed through clear, structured, living requirements.

Specifications First

Define what you want before a line of code exists

Living Documents

Specs evolve continuously — not written once and forgotten

Human Review

Every phase needs explicit approval before moving on

Single Source of Truth

One spec guides both humans and AI agents

Docs Live With Code

Specs and implementation share the same repo

THE TOOLKIT

GitHub Spec Kit

An open-source toolkit that brings Spec-Driven Development to life — a simple CLI plus structured templates that work with the agent you already use.

106K+	30	105	200+
GitHub stars	Agent integrations	Community extensions	Contributors

- Open source & agent-agnostic by design
- A Python CLI plus battle-tested templates
- Drops into VS Code or your favorite AI tool

First released Sept 2025 by GitHub — now a fast-moving community platform.

SINCE OUR LAST TALK

What's New in Spec Kit

The --integration system

The old --ai flags are gone. Pick your agent with --integration copilot | claude | codex | gemini …

Extensions & presets

A whole ecosystem: 105 extensions & 22 presets let you add capabilities or reshape the workflow.

Self-management built in

specify self check and specify self upgrade keep the CLI current — no reinstall dance.

Skills mode + 30 agents

Install Spec Kit as agent skills (--skills), and switch freely across 30+ supported agents.

UNDER THE HOOD

The Specify CLI & Project Structure

The Specify CLI

- Bootstraps your project scaffolding
- Downloads the official SDD templates
- Wires up your chosen AI agent
- Installs the /speckit.* slash commands

Your project after init

```
.specify/
   memory/constitution.md
   templates/  (spec, plan, tasks)
   scripts/
   extensions/   presets/
specs/
   001-inkwell/
      spec.md  plan.md  tasks.md
.github/  .claude/  (agent commands)
```

SETUP

Getting Started in Two Steps

Prerequisites

- uv (or pipx)
- Python 3.11+
- Git
- An AI agent

1 Install the Specify CLI

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git@vX.Y.Z
```

2 Initialize your project & pick your agent

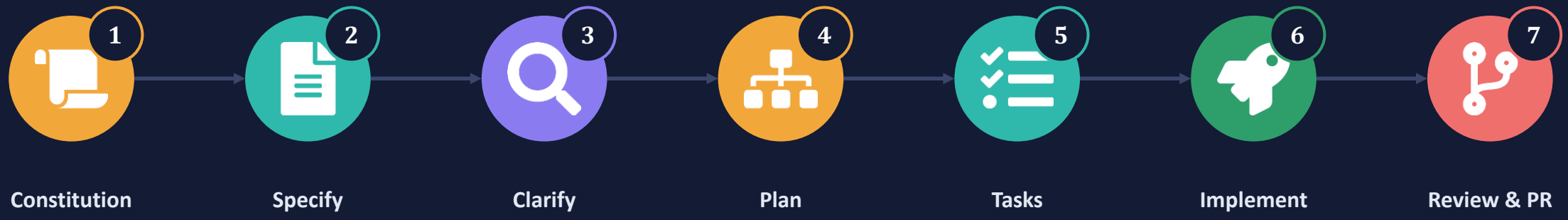
```
specify init inkwell --integration claude
```

Then open your agent and the /speckit. commands are ready to go.*

THE WORKFLOW

From Intent to Implementation

Seven steps take InkWell from a single sentence to shipped code:



Quality gates along the way: `/speckit.clarify` before planning, `/speckit.checklist` and `/speckit.analyze` before you implement.

OUR RUNNING DEMO

Meet InkWell

A novelist's writing studio — the app we'll build end-to-end with Spec Kit.

Chapter drafting

Distraction-free editor with autosave

Character & plot tracker

Keep your cast and threads straight

Word-count goals

Daily targets and streaks that motivate

Local-first & private

Manuscripts stay on the writer's device

Same story, every phase: watch InkWell grow from a sentence into a working app.

STEP 1 · /SPECKIT.CONSTITUTION

Set the Ground Rules

```
/speckit.constitution
```

The foundation

A set of non-negotiable principles the agent must honor in every later phase — quality bars, tech constraints, UX and accessibility standards. Stored in .specify/memory/constitution.md.

InkWell's constitution

- A writer's words are never lost — autosave + local-first storage
- Manuscripts stay private; nothing leaves the device without consent
- .NET everywhere, with tests for every feature
- Meet WCAG 2.1 AA — writing tools must be accessible

STEP 2 · /SPECKIT.SPECIFY

```
/speckit.specify
```

Describe What to Build

The what & the why

Focus on outcomes, not tech. Capture goals, target users, core features, and success criteria. Spec Kit creates a feature branch (001-inkwell) and a spec.md full of user stories.

InkWell's spec prompt

“Build an app for novelists to draft chapters and organize them into a manuscript. Writers track characters and plot threads, set daily word-count goals, and write in a distraction-free editor. Everything works offline and saves locally.”

STEP 3 · /SPECKIT.CLARIFY

```
/speckit.clarify
```

Close the Gaps Before You Plan

Structured Q&A

A sequential, coverage-based interview surfaces what the first draft missed and records answers in a Clarifications section. Run it before /speckit.plan to cut rework. (Formerly /quizme.)

What InkWell needs to clarify

- Rich text or Markdown for chapter content?
- How do we resolve edits made on two devices offline?
- Which export formats — EPUB, PDF, DOCX?
- Is there a hard limit on manuscript or chapter size?

STEP 4 · /SPECKIT.PLAN

```
/speckit.plan
```

Choose the How

The technical plan

Now name your stack. The agent reads the spec + constitution and produces plan.md, research.md, data-model.md, contracts/, and quickstart.md — architecture, data, and APIs in detail.

InkWell's plan prompt

“Use .NET MAUI, with a Clean Architecture design, with a minimal library footprint. Store manuscripts encrypted in a local SQLite / IndexedDB database — nothing is uploaded. Keep the editor component framework-agnostic and fully keyboard-navigable.”

QUALITY GATES

Unit Tests for Your English

Two optional commands catch problems while they're still cheap — before any code is written.

`/speckit.checklist`

Generate custom quality checklists that validate your requirements for completeness, clarity, and consistency — “unit tests for English.”

For InkWell: does every user story define success? Are offline behaviors specified?

`/speckit.analyze`

Cross-artifact consistency and coverage analysis. Run it after /speckit.tasks and before /speckit.implement to catch drift between spec, plan, and tasks.

For InkWell: does the plan actually cover the export formats the spec promised?

STEP 5 · /SPECKIT.TASKS

```
/speckit.tasks
```

Break It Into Actionable Work

From plan to a build list

- Tasks grouped by user story — each ships independently
- Ordered by dependency (models → services → endpoints)
- Parallel-safe work flagged with [P]
- Exact file paths on every task
- Test-first ordering when TDD is requested

Bonus: /speckit.taskstoissues

Turn the generated task list into tracked GitHub issues in one command — so the whole team can pick up InkWell's work.

```
tasks.md
  [P] T003  models/manuscript.ts
  [P] T004  models/chapter.ts
      T007  editor/autosave.ts
      T012  export/epub.ts
```

STEP 6 · /SPECKIT.IMPLEMENT

Let the Agent Build InkWell

```
/speckit.implement
```

Validates prerequisites

Constitution, spec, plan, and tasks all present

Parses tasks.md

Reads the ordered breakdown and dependencies

Executes in order

Respects sequence and [P] parallel markers

Follows TDD & the plan

Tests first where you asked; principles enforced

The result: a working InkWell — chapters, characters, goals, and a distraction-free editor — built from specs, not vibes.

STEP 7 · YOU'RE STILL THE AUTHOR

Review, Verify, Merge

Human oversight is the point

Spec Kit gets you a working draft fast — but you're still the author. Read the code, run it, catch what CLI logs won't show (browser console errors, UX rough edges), and paste issues back to the agent.

1. **Review the diff**
   Does the code match the spec and constitution?
2. **Run & QA**
   Exercise InkWell — write a chapter, hit a word goal
3. **Sign off**
   Approve, open a PR with a clear description
4. **Merge**
   Ship it — with docs living beside the code

NEW IN 2026

Make Spec Kit Your Own

Extensions

ADD new capabilities

New commands, phases, and integrations. CI Guard, Architecture Guard, Jira sync — or an InkWell “manuscript-export” command.

```
specify extension add <name>
```

Presets

CHANGE how it works

Reshape templates, terminology, and standards without new commands. Enforce a house style guide on every spec and plan.

```
specify preset add <name>
```

Or swap the whole process: AIDE, Canon, Product Forge, MAQA — community presets that redefine SDD end-to-end. 105 extensions, 22 presets, and counting.

NO LOCK-IN

Works With 30+ AI Agents

Spec Kit is agent-agnostic. Switch freely with one flag — run `specify integration list` to see them all.

CLI / Terminal

Claude Code
Codex CLI
Gemini CLI
Qwen Code
opencode
Goose
Mistral Vibe
Forge

IDE Assistants

GitHub Copilot
Cursor
Windsurf
Kiro
Tabnine
Qoder
Roo Code
Trae

Escape Hatch

Generic integration — bring any agent

Skills mode:

--integration-options="--skills" installs agent skills

FROM THE TRENCHES

Best Practices & Tips

Do

- ✓ Start with the MVP — one small spec first
- ✓ Review each phase before moving on
- ✓ Use specific, measurable success criteria
- ✓ Keep a human supervising the agent
- ✓ Iterate — specs are living documents

Don't

- ✗ Skip the constitution — it's the foundation
- ✗ Let the AI run unsupervised
- ✗ Treat specs as fixed waterfall documents
- ✗ Over-engineer from the very start
- ✗ Cram everything into one giant spec

THE BIGGER PICTURE

The Spec-Driven Ecosystem

Spec Kit isn't alone — structured, spec-first development is becoming the default for AI-assisted work.

Amazon Kiro

Agentic IDE with spec-driven development built in

Tessl Framework

A spec-driven registry for reliable AI coding agents

Alt SDD processes

AIDE, Canon & Product Forge reimagine the workflow

MAQA

Multi-agent orchestration with QA gates

IN SUMMARY

Five Things to Remember

1	**Vibe coding doesn't scale**	Clear specs are essential for reliable AI-assisted development
2	**Spec Kit gives you structure**	A proven path from constitution to implementation — and beyond
3	**It's more powerful in 2026**	Extensions, presets, and skills mode make it truly yours
4	**Works with your tools**	30+ agents, no lock-in — Copilot, Claude, Cursor, and more
5	**You're still the author**	Start small, iterate often, and keep a human in the loop

GO DEEPER

Resources & Links

Official

- github.com/github/spec-kit
- github.github.io/spec-kit
- Quickstart & installation guide

Extend It

- Integrations reference
- Community extensions & presets
- Walkthroughs & “friends”

Learn

- Video overview (YouTube)
- den.dev/blog — Den Delimarsky
- spec-driven.md deep dive

Thank You!

Questions?

BlueSky @barretblake.dev

Web barretblake.dev

YouTube youtube.com/@barretcodes

LinkedIn linkedin.com/in/barretblake

NimblePros blog.nimblepros.com

`github.com/github/spec-kit` · `github.github.io/spec-kit`